An Empirical Evaluation of Synchronization Mechanisms in Shared-Memory Parallel Systems

Pasupuleti T A N S V Vignesh
IIIT-Bangalore
IMT2023003

Kunda Sri Krishna Chaitanya
IIIT-Bangalore
IMT2023039

***Abstract*—Synchronization primitives are a fundamental building block of concurrent applications, yet their comparative behavior across workloads, thread counts, and hardware configurations is rarely studied in a unified experimental framework. This paper evaluates four synchronization primitives—`std::mutex`, POSIX Semaphore, Test-and-Set SpinLock (TAS), and the MCS queue lock—across six dimensions: scalability, critical-section contention sensitivity, false-sharing overhead, fairness, tail-latency distribution, and throughput saturation under oversubscription. We apply the Gini coefficient to per-thread lock acquisitions as a single, scale-independent measure in $[0, 1]$ that quantifies imbalance and potential starvation. In addition to microbenchmarks,we validate the findings on two representative concurrent application patterns, a striped-lock concurrent hash table and a bounded producer-consumer queue, under different access patterns. The complete benchmark suite is publicly available at https://github.com/Vignesh1131321/SSP_Final_Project.git.**

***Index Terms*—Synchronization Primitives, Mutual Exclusion, MCS Lock, SpinLock, Semaphore, Gini Coefficient, Lock Fairness, False Sharing, Concurrent Data Structures, Benchmarking, Multicore**

I. Introduction

The correctness and performance of every multi-threaded application ultimately rests on its choice of synchronization primitive. From an operating-system scheduler to a web-server connection pool, the lock that guards a critical section determines throughput, latency, and fairness for all competing threads. Despite its importance, practitioners often choose `std::mutex` across workloads, largely due to the lack of clear, reproducible comparisons under diverse conditions.

This paper addresses that gap through a multi-experiment benchmark suite implemented in C++17. We deliberately select four primitives that span the design space along three axes—memory model (kernel vs. user-space), waiting strategy (blocking vs. spinning), and fairness guarantee (none, weak, FIFO)—so that differences in experimental outcomes can be traced back to specific design decisions rather than incidental implementation details. The four primitives are: `std::mutex` (hybrid user/kernel, futex-based, weakly fair), POSIX Semaphore (pure kernel, weakly fair), Test-and-Set SpinLock (pure user-space, no fairness guarantee), and MCS Lock [2] (pure user-space, FIFO-fair, local spinning).

Our principal contributions are as follows.

1) A reproducible benchmark suite covering six experimental dimensions—scalability, contention, false sharing, fairness, tail latency, and throughput saturation.
2) Application of the *Gini coefficient* as a fairness metric for lock benchmarking, in comparison with Jain's fairness index commonly used in literature, showing that Gini provides a more sensitive and reliable measure of fairness across varying thread counts and lock behaviors.
3) Two representative concurrent application patterns (a concurrent hash table and a bounded producer–consumer queue) under six access-pattern scenarios each, bridging the gap between microbenchmarks and application-level performance.

The remainder of this paper is organised as follows. Section II surveys related work. Section III describes the benchmark methodology. Section IV presents experimental results. Section V concludes the paper.

II. Related Work

A. MCS vs. TAS SpinLock

Queue-based locks such as MCS [2] avoid contention on a single shared memory location by letting each thread spin on a private node, which reduces coherence traffic and improves scalability. In contrast, the Test-and-Set spinlock repeatedly updates one shared cache line, so it is a useful baseline for exposing cache-thrashing under contention. We therefore use MCS as the queue-lock reference and TAS as the simplest contention-heavy baseline.

B. Gini Coefficient as a Fairness Metric

Fairness in lock benchmarking is often reported with max/min ratio or coefficient of variation, but both are limited for cross-experiment comparison. The *Gini coefficient* [7] resolves this by providing a bounded, scale-independent measure of distributional inequality. For n threads with per-thread acquisition counts $x_1, x_2, \ldots, x_n$, it is defined as:

$$G = \frac{\sum_{i=1}^{n}\sum_{j=1}^{n}|x_i - x_j|}{2n\sum_{i=1}^{n} x_i} \tag{1}$$

$G = 0$ indicates perfect equality and $G = 1$ indicates complete starvation. The metric is scale-independent, so it remains

comparable even when throughput differs substantially across primitives. For our four primitives, MCS should be closest to zero, TAS highest, and `std::mutex`/Semaphore intermediate.

C. Statistical Benchmarking Methodology

Following Georges et al. [10], each configuration uses warmup phases, repeated trials, and confidence intervals. We use Welford's algorithm [8] for numerically stable online mean and variance, and reservoir sampling for tail latencies as recommended by Fleming and Wallace [12].

III. Benchmark Methodology

A. Primitives Selected

We evaluate four primitives chosen to maximally span the design space along three axes: memory model, waiting strategy, and fairness guarantee. Table I summarises their key properties.

TABLE I
Synchronization Primitives Under Study

Primitive	Space	Waits	Fairness	Traffic
`std::mutex`	User+Kernel	Block	Weak	$O(1)$
POSIX Semaphore	Kernel	Block	Weak	$O(1)$
SpinLock (TAS)	User	Spin	None	$O(P)$
MCS Lock	User	Spin	FIFO	$O(1)$

B. Experimental Dimensions

We evaluate six microbenchmark dimensions:

1) **Scalability** — Throughput and mean latency vs. thread count at zero critical-section work.
2) **Contention** — Throughput vs. busy-loop critical-section size at fixed thread count.
3) **False Sharing** — Throughput ratio of shared-line vs. padded counters, with and without CPU pinning.
4) **Fairness** — Gini coefficient of per-thread acquisition counts at 2, 4, and 8 threads.
5) **Latency Distribution** — P50/P95/P99/P99.9 latency from reservoir-sampled distributions.
6) **Throughput Saturation** — Throughput sweep from 1 thread to $2\times$ hardware concurrency.

C. Statistical Validity

Each configuration is repeated multiple times with a warmup phase to stabilise CPU frequency scaling, TLB warm-up, and branch prediction. Aggregate statistics use Welford's algorithm [8] and 95% confidence intervals use the Student t-distribution with $n-1$ degrees of freedom. Percentile latencies are computed from reservoir-sampled distributions to bound memory usage at $O(k)$ for a fixed reservoir size k.

IV. Experimental Results: Microbenchmarks and Workloads

A. Experiment 1: Lock Scalability and CPU Utilization

1) Setup: All four primitives were tested at thread counts $T \in \{1, 2, 4, 8, 16, 32\}$. The system provides 12 logical cores, so $T > 12$ represents oversubscription. The critical section performs zero work to isolate lock overhead for throughput analysis. Additionally, a separate trial was conducted with the critical section increased to 2000 busy-loop cycles to measure CPU utilization, ensuring sufficient duration to track process-wide CPU usage via `/proc/stat`. Each configuration ran for 5 s with a 2 s warmup, repeated 5 times, and threads were pinned to distinct logical cores (for throughput) and left to the OS scheduler (for CPU utilization).

2) Results: Figure 1 shows throughput as thread count increases. SpinLock achieves the highest throughput at $T = 1$ due to minimal user-space overhead. However, its performance degrades rapidly with increasing threads because repeated test-and-set operations generate high cache-coherence traffic.

MCS lock scales better up to moderate thread counts ($T \leq 8$) by ensuring that each thread spins on a private cache line, reducing contention on shared memory. Beyond hardware concurrency ($T > 12$), its performance drops due to oversubscription, where lock handoff is delayed by thread scheduling.

In contrast, `std::mutex` and Semaphore exhibit a plateau after $T \approx 4$–8. At this point, contention causes threads to block and be managed by the OS scheduler. As the critical section is inherently serialized, additional threads do not increase throughput, resulting in a stable but capped performance dominated by context-switch overhead.

Figure 2 illustrates the normalized per-core CPU utilization across varying thread counts with a 2000-cycle critical section. At lower thread counts ($T \leq 8$), all primitives exhibit relatively similar CPU utilization, increasing linearly as more cores are engaged. However, under oversubscription ($T \geq 16$), a stark divergence occurs. The SpinLock's utilization sharply rises to plateau near 1.0 (100% utilization per core), as waiting threads continuously spin in user space burning CPU cycles. In contrast, the sleeping locks (`std::mutex` and Semaphore) experience a significant decline in CPU utilization. Because the critical section is sufficiently long (2000 cycles), blocked threads have enough time to be descheduled by the OS scheduler, leaving the remaining cores largely idle rather than actively thrashing. Note that the MCS lock drops at $T \geq 16$ because its execution is explicitly skipped to prevent deadlock under oversubscription without CPU pinning; its reported utilization merely reflects baseline system noise during immediate termination.

Key finding: *MCS lock provides the best scalability up to hardware concurrency, while SpinLock is only optimal at very low thread counts. Blocking primitives trade high performance for stability, achieving consistent throughput under high contention. Furthermore, SpinLock fully saturates CPU*

resources regardless of contention, whereas sleeping primitives efficiently relinquish CPU time when oversubscribed.

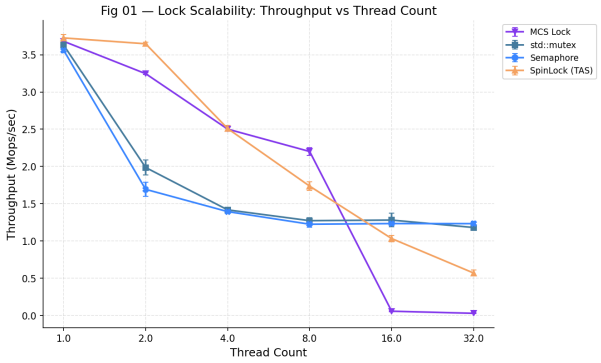

Fig. 1. Throughput vs thread count for all synchronization primitives.

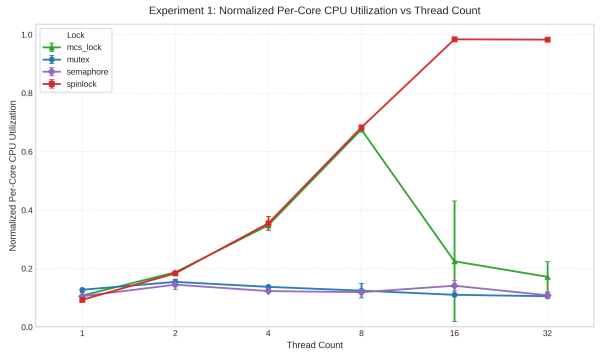

Fig. 2. Normalized per-core CPU utilization vs thread count (CS = 2000 cycles).

B. Experiment 2: Contention / Critical-Section Size

1) Setup: This experiment fixes $T = 8$ and varies critical-section (CS) work over $\{0, 100, 1000, 10{,}000\}$ busy-loop cycles. The setup otherwise matches Experiment 1: 2 s runs, 1 s warmup, five repetitions, and pinned threads.

2) Results: Figure 3 shows throughput as CS work increases. At CS = 0, MCS Lock achieves the highest throughput due to FIFO ordering and reduced cache-coherence traffic. In contrast, SpinLock (TAS) suffers from contention-induced spinning, leading to coherence overhead even for short CS.

`std::mutex` and Semaphore perform similarly at small CS sizes but incur higher overhead due to blocking and wake-up costs. However, they avoid excessive CPU spinning, making them more stable than SpinLock under contention.

As CS work increases, throughput drops for all primitives since execution becomes dominated by serialized critical sections. The relative differences between locks diminish at large CS sizes as lock overhead becomes negligible compared to CS execution time.

Key finding: *MCS provides the best scalability under contention, SpinLock is penalized by coherence overhead, and blocking primitives trade higher overhead for better stability. At large CS sizes, all approaches converge due to serialization.*

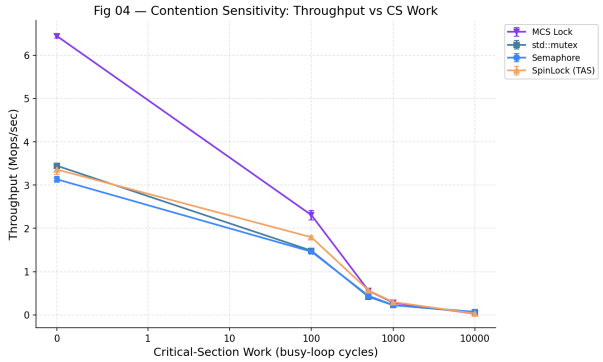

Fig. 3. Throughput vs critical-section size at $T = 8$ threads.

C. Experiment 3: False Sharing

1) Setup: This experiment measures the false-sharing penalty by comparing padded and false-shared counter layouts under SMT ON and SMT OFF modes. The reported metric is the slowdown factor, defined as padded throughput divided by false-shared throughput, at thread counts $T \in \{1, 2, 3, 4, 6, 8, 10, 12\}$. Each configuration uses the same 2 s run length, 1 s warmup, five repetitions, and pinned threads as the previous experiments.

2) Results: Figure 4 shows that false sharing consistently imposes a throughput penalty above the $1.0\times$ baseline once more than one thread is active. Under both SMT ON and SMT OFF, the slowdown rises to roughly $2\times$ at moderate and high thread counts, with the largest penalties appearing around $T = 8$ to 12. SMT OFF exposes the cache-line ping-pong more clearly, while SMT ON slightly smooths the curve but does not remove the effect.

Key finding: *False sharing reduces throughput by about $2\times$ under contention, and the effect persists regardless of SMT mode. Padding shared counters to separate cache lines is a simple and effective way to recover performance.*

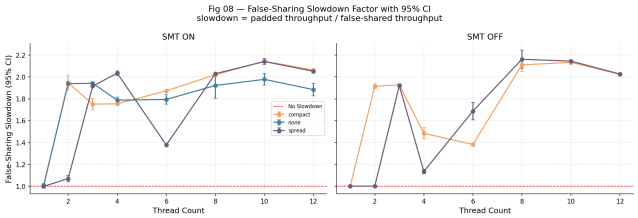

Fig. 4. False-sharing slowdown factor with 95% confidence intervals. The slowdown is defined as padded throughput divided by false-shared throughput.

D. Experiment 4: Fairness (Gini vs. Jain)

1) Setup: Fairness is evaluated using Jain's fairness index and the Gini coefficient on per-thread lock acquisition counts at $T \in \{2, 4, 8\}$ under the same experimental conditions as earlier sections. To test metric sensitivity, we construct four synthetic distributions: **REF** (perfect equality: `[100, 100, 100, 100, 100, 100, 100, 100]`), **JF-1** (mild skew, 2.5× gap: `[120,`

`120, 120, 120, 48, 48, 48, 48]`), **JF-2** (moderate skew, 3× gap: `[150, 150, 150, 150, 50, 50, 50, 50]`), and **JF-3** (single starved thread: `[115, 115, 115, 115, 115, 115, 115, 5]`). The JF cases target the "Jain divergence zone"—where Jain reports high fairness while Gini still detects true inequality, comparing the metrics before applying them to real locks.

2) Results: Our evaluation of the synthetic distributions reveals a critical "divergence zone" ($J \geq 0.8$, $G \geq 0.15$) where Jain's index appears optimistic despite measurable imbalance. This validates the metric divergence: **REF** evaluates ideally at perfect equality, while the skewed distributions **JF-1** and **JF-2** fall directly into the divergence zone. Because Jain's formula relies on the sum of squares, the structural inequality of the bimodal 2× to 3× gaps in JF-1 and JF-2 is mathematically diluted, yielding deceptively high fairness scores. In contrast, Gini calculates pairwise absolute differences, remaining highly sensitive to the systematic skew. **JF-3** narrowly misses the zone because a single starved thread does not sufficiently skew the pairwise average of 8 threads to push Gini over the 0.15 threshold, although Jain still reports near-perfect fairness.

Figure 5 shows fairness trends across primitives as thread count increases. At low thread counts, all locks exhibit near-uniform acquisition due to limited contention. As contention increases, SpinLock (TAS) degrades significantly because threads repeatedly compete on a shared cache line, allowing recently successful threads to reacquire the lock quickly while others are delayed, leading to imbalance. In contrast, MCS maintains strong fairness due to its queue-based design, which enforces FIFO ordering and prevents overtaking. Blocking primitives such as std::mutex and semaphores remain relatively fair, as the OS scheduler redistributes CPU time among waiting threads, although minor imbalance can still arise from scheduling variability.

Key finding: *Fairness varies significantly across lock designs under contention: MCS and blocking primitives maintain near-uniform acquisition, while SpinLock exhibits increasing imbalance as thread count rises. Jain's index tends to saturate near high fairness and can hide moderate skew imbalance, whereas the Gini coefficient provides better sensitivity to imbalance, finer discrimination and more clearly reflects skew and starvation. Overall as a result, Gini is a more reliable standalone metric for fairness evaluation, especially under contention.*

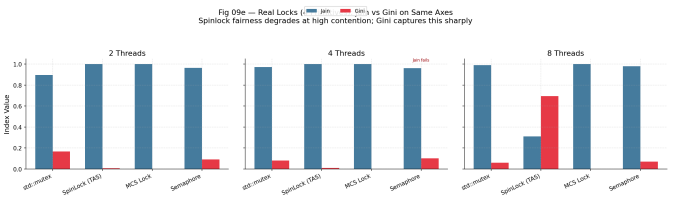

Fig. 5. Real locks: Jain (blue) and Gini (red) on the same axes for 2, 4, and 8 threads; Gini highlights fairness degradation of spinlocks at high contention.

E. Experiment 5: Latency Distribution and Performance Counter Analysis

1) Setup: We evaluate (i) latency distribution (P50, P95, P99, P99.9) at $T = 8$ using reservoir sampling over 5 runs, and (ii) cache and branch MPKI across $T \in 2, 4, 8$. Hardware counters are collected via `perf` (cache misses, branch misses, instructions) to relate tail latency to microarchitectural overheads.

2) Results and Interpretation:

a) Latency Distribution.: Figure 6 shows that MCS Lock has the most concentrated distribution with minimal tail latency due to FIFO ordering and single handoff per acquisition. SpinLock (TAS) exhibits the widest distribution and highest tail latency, caused by repeated failed atomic operations under contention. `std::mutex` and Semaphore show heavy-tailed behaviour due to kernel blocking and scheduling delays.

b) MPKI Analysis.: Figure 7 reveals three behaviours. MCS Lock attains the highest cache MPKI (peaking at ≈22.5) due to pointer chasing, yet maintains low latency since these misses are local and non-amplifying. SpinLock shows slightly lower MPKI but significantly higher latency because misses occur on a shared cache line, triggering coherence invalidations and retry loops. In contrast, blocking primitives have low MPKI since threads sleep under contention, but incur high tail latency due to context-switch and wake-up delays. Branch MPKI follows a similar trend, with higher values for spin-based locks due to polling loops.

Key finding: *MPKI does not directly determine latency. MCS Lock achieves low tail latency despite high MPKI by avoiding contention amplification, whereas SpinLock suffers from expensive, shared misses. Blocking primitives minimise hardware overhead but shift cost to OS scheduling, resulting in low MPKI but high latency. Thus, latency is governed by how misses interact with contention, not merely their frequency.*

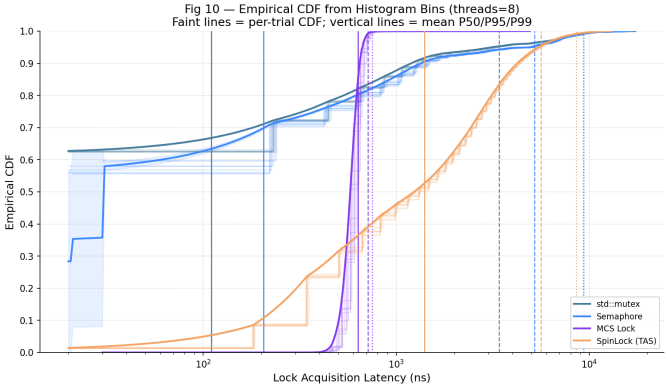

Fig. 6. Empirical CDF of lock-acquisition latency at 8 threads; vertical lines mark P50, P95, P99, P99.9 percentiles.

F. Experiment 6: Throughput Saturation

1) Setup: This experiment sweeps thread count from 1 to 16 and reports throughput under two critical-section sizes: 0 cycles and 500 cycles. The vertical shaded band marks the

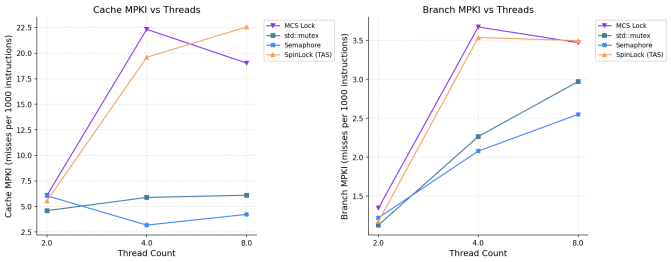

Fig. 7. Cache and Branch MPKI (misses per 1000 instructions) vs. thread count

oversubscribed region beyond the system's 12 logical CPUs. All runs use the same warmup, repetition count, and pinning policy as the earlier experiments.

2) Results: Figure 8 shows the saturation curves under two regimes. With a zero-cycle critical section, throughput peaks at low thread counts (2–4 threads) and then declines as contention increases. SpinLock and MCS achieve the highest throughput initially due to low overhead, but both degrade rapidly as threads compete for the same cache line. In contrast, `std::mutex` and Semaphore exhibit lower peak throughput but stabilize under contention, maintaining steady performance beyond 8 threads.

With 500 cycles of critical-section work, throughput differences across primitives are reduced, and saturation occurs more gradually. MCS performs best up to moderate thread counts (8 threads), benefiting from reduced contention through queueing. However, once the system becomes oversubscribed ($>$ 12 threads), blocking primitives (`std::mutex` and Semaphore) are more robust, while spinning-based approaches degrade due to increased scheduling and coherence overheads.

Across both workloads, throughput saturates well before the hardware limit and declines in the oversubscribed region, demonstrating that adding threads beyond available cores reduces overall system throughput.

Key finding: *Throughput saturates at modest thread counts due to contention, and declines under oversubscription. Spinning locks achieve higher peak performance at low contention, while blocking primitives provide more stable performance when the number of runnable threads exceeds available hardware resources.*

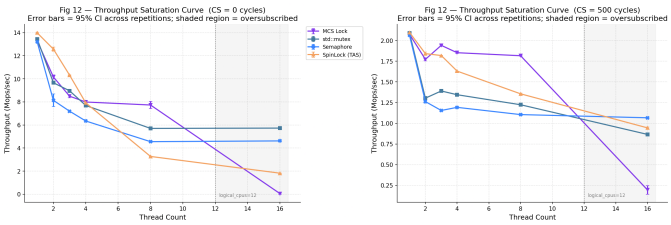

Fig. 8. Throughput saturation curves at 0 and 500 critical-section cycles. The shaded region indicates oversubscription beyond 12 logical CPUs.

G. Representative Concurrent Workloads

1) Setup: To validate microbenchmark trends in application-level settings, we evaluate two concurrent workloads: a hash table (coarse mutex, striped RW-lock, lock-free CAS) and a bounded producer-consumer queue (mutex queue, semaphore queue, lock-free ring). Throughput is measured across thread counts from 1 to 12 with 10 repetitions per point.

2) Results: Figures 9 and 10 show the balanced scenarios while Figure 11 shows bursty scenario in producer-consumer queue.

For the **hash table**, the lock-free implementation scales nearly linearly with thread count and achieves the highest throughput. The striped RW-lock improves over the coarse mutex by reducing contention via partitioning, but begins to plateau at higher thread counts due to lock coordination overhead. The coarse mutex performs worst, with throughput degrading as threads increase, indicating strong serialization bottlenecks.

For the **producer–consumer queue**, the lock-free ring buffer achieves the highest throughput and scales well up to moderate thread counts before slightly tapering, reflecting reduced contention but eventual resource saturation. In contrast, both mutex and semaphore-based queues degrade rapidly beyond 2–4 threads and flatten at low throughput levels due to blocking, context-switching overhead, and contention on shared queue state.

Across the remaining scenarios, similar trends persist with varying intensity. In **read-dominant** workloads, the performance gap widens in favor of lock-free, as striped RW-lock still incurs locking overhead despite allowing concurrent readers, while coarse mutex remains a bottleneck. **Write-dominant** workloads increase contention for all approaches, but lock-free retains its advantage by avoiding blocking; RW-lock degrades relative to read-heavy cases due to exclusive write locking. Under **hotspot and small-buffer** conditions, localized contention severely impacts blocking approaches through serialization, while lock-free shows reduced but still superior gains due to increased CAS retries. **Bursty backpressure** lowers overall throughput across all implementations, as threads stall on full/empty conditions rather than lock contention, narrowing the lock-free advantage. In **delete-heavy and fan-in/fan-out** scenarios, lock-free remains superior despite slightly elevated retry costs, while mutex and semaphore queues suffer from contention asymmetry and increased synchronization overhead.

Overall, the results consistently show that **contention on shared critical paths is the dominant factor limiting scalability**. Lock-free designs mitigate this by avoiding blocking and enabling fine-grained progress, whereas lock-based approaches eventually suffer from coordination overhead and serialization.

H. Summary

Table II presents a qualitative comparison across the four primitives across all microbenchmark dimensions.

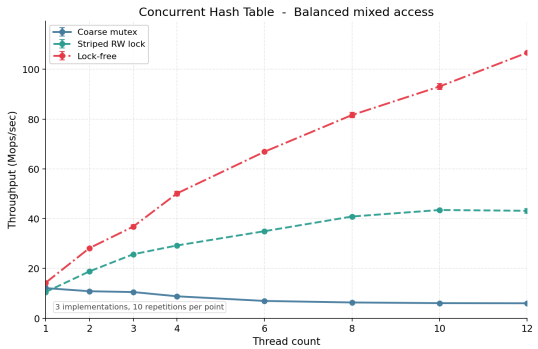

Fig. 9. Concurrent hash table throughput under balanced mixed access.

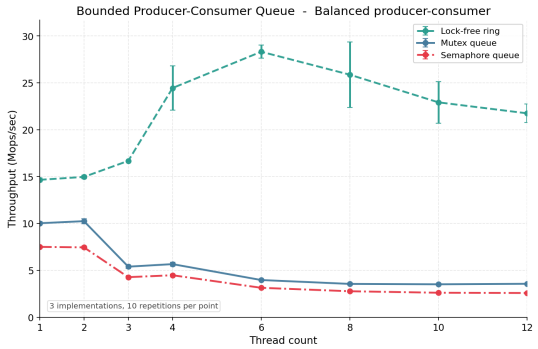

Fig. 10. Bounded producer-consumer queue throughput under balanced 1:1 producer-consumer load.

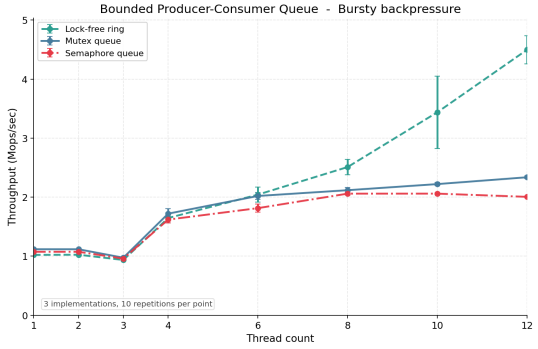

Fig. 11. Bounded producer–consumer queue under bursty backpressure.

TABLE II
COMPACT QUALITATIVE SUMMARY ACROSS ALL MICROBENCHMARK DIMENSIONS

Dimension	**Mutex**	**Sem.**	**Spin**	**MCS**
Scalability (Exp. 1)	Med.	Med	Poor	**Best**
Short CS throughput (Exp. 2)	Med.	Med	Med	**Best**
Fairness under contention (Exp. 4)	Good	Good	Poor	**Best**
Tail-latency behavior (Exp. 5)	Poor	Poor	Poor	**Best**
Oversubscription stability (Exp. 6)	Good	Good	Poor	Med.

V. CONCLUSION

This paper presents a comprehensive empirical evaluation of four synchronization primitives across six microbenchmark dimensions and two representative workloads. Our results demonstrate that contention on shared critical paths is the dominant factor limiting scalability: MCS Lock provides the best throughput and fairness at moderate thread counts, blocking primitives remain robust under oversubscription, and SpinLock degrades rapidly under contention due to coherence overhead.

The Gini coefficient proves superior to traditional fairness metrics, and false-sharing imposes a consistent $2\times$ penalty independent of lock choice. Representative workload validation confirms these trends: lock-free algorithms scale best under high contention, while traditional locks remain competitive in lightly-contended scenarios. These results provide practitioners with concrete guidance for synchronization primitive selection based on contention level, fairness requirements, and hardware concurrency constraints.

REFERENCES

[1] O. Kode and T. Oyemade, "Analysis of synchronization mechanisms in operating systems," arXiv:2409.11271 [cs.OS], Sep. 2024.

[2] J. M. Mellor-Crummey and M. L. Scott, "Algorithms for scalable synchronization on shared-memory multiprocessors," *ACM Trans. Comput. Syst. (TOCS)*, vol. 9, no. 1, pp. 21–65, Feb. 1991.

[3] T. E. Anderson, "The performance of spin lock alternatives for shared-memory multiprocessors," *IEEE Trans. Parallel Distrib. Syst.*, vol. 1, no. 1, pp. 6–16, Jan. 1990.

[4] T. S. Craig, "Building FIFO and priority-queueing spin locks from atomic swap," Tech. Rep. TR 93-02-02, Univ. of Washington, Feb. 1993.

[5] P. Magnusson, A. Landin, and E. Hagersten, "Queue locks on cache coherent multiprocessors," in *Proc. 8th Int. Parallel Processing Symp. (IPPS)*, pp. 165–171, Apr. 1994.

[6] M. L. Scott and W. N. Scherer, "Scalable queue-based spin locks with timeout," in *Proc. 8th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, pp. 44–52, 2001.

[7] C. Gini, "Measurement of inequality of incomes," *The Economic Journal*, vol. 31, no. 121, pp. 124–126, Mar. 1921.

[8] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, Aug. 1962.

[9] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.

[10] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pp. 57–76, 2007.

[11] U. Drepper, "Futexes are tricky," Red Hat Inc., Tech. Rep., ver. 1.2, 2011. [Online]. Available: http://www.akkadia.org/drepper/futex.pdf

[12] P. J. Fleming and J. J. Wallace, "How not to lie with statistics: The correct way to summarize benchmark results," *Commun. ACM*, vol. 29, no. 3, pp. 218–221, Mar. 1986.

[13] D. Vyukov, "Bounded MPMC queue," 2010. [Online]. Available: https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue